IT6070 EMERGING TOPICS IN IT · ASSIGNMENT II

Prescription Reader

Reading handwritten prescriptions with Amazon Nova – what was built, how it works, and what the tests showed

In one paragraph

A pharmacist at Dulnitha Pharmacy, Kottawa spends many hours a day reading handwritten prescriptions. This project is a web app that takes a photo (or PDF) of a prescription, asks an AI model on Amazon Web Services – **Amazon Nova 2 Lite** – to read it, and shows the result as an editable table next to the photo. The pharmacist checks and corrects every item, then confirms the list. If the AI cannot help, the app switches to typing the items in by hand. The app is fast (about 2.5 seconds per prescription) and cheap (about LKR 0.34 per prescription), and it flags its own mistakes well – but on dense handwriting it reads only about half of the drug names correctly, so the pharmacist must always check.

Contents

1. The problem
2. The idea and how it works
3. Technologies used – and why
4. The app, page by page
5. Safety and privacy
6. Testing and results
7. Cost
8. What is done and what is left
9. Glossary – simple meanings of technical words

Student: U.W.D.P. Amarasinghe (MS25903874) · Professional: "Pharmacist A" (pseudonym), Dulnitha Pharmacy, Kottawa · Status as of 06/10/2026 · Screenshots use a **synthetic** sample prescription and a simulated AI answer, so no real patient data appears.

1. The problem

"Pharmacist A" is a registered pharmacist at an independent community pharmacy in Kottawa. In a discussion on 30/09/2026 they described the daily workload (these are the pharmacist's own estimates):

Fact	Number
Prescriptions per day	about 200
Handwritten	70–80%
Time to read, check and match one prescription to stock	2–3 minutes
Staff time spent on this every day	about 5–8 hours
Prescriptions that need a colleague or the doctor to confirm	about 3%

Doctors' handwriting is hard to read, and doctors use short forms such as `bd` (twice a day), `nocte` (at night) or `5/7` (five days). Misreading a drug name or dose can harm a patient.

What the app must do (requirements)

- **R1 – Read the prescription.** From a phone photo, list each item's drug name, strength, dose, frequency and duration in an editable list, expand the short forms, mark the fields it is unsure about, and do it within 30 seconds.
- **R2 – Check before dispensing.** Compare the confirmed list with the pharmacy's stock and a list of drug interactions, and produce a checklist with warnings and in-stock alternatives. No patient name is sent or stored.

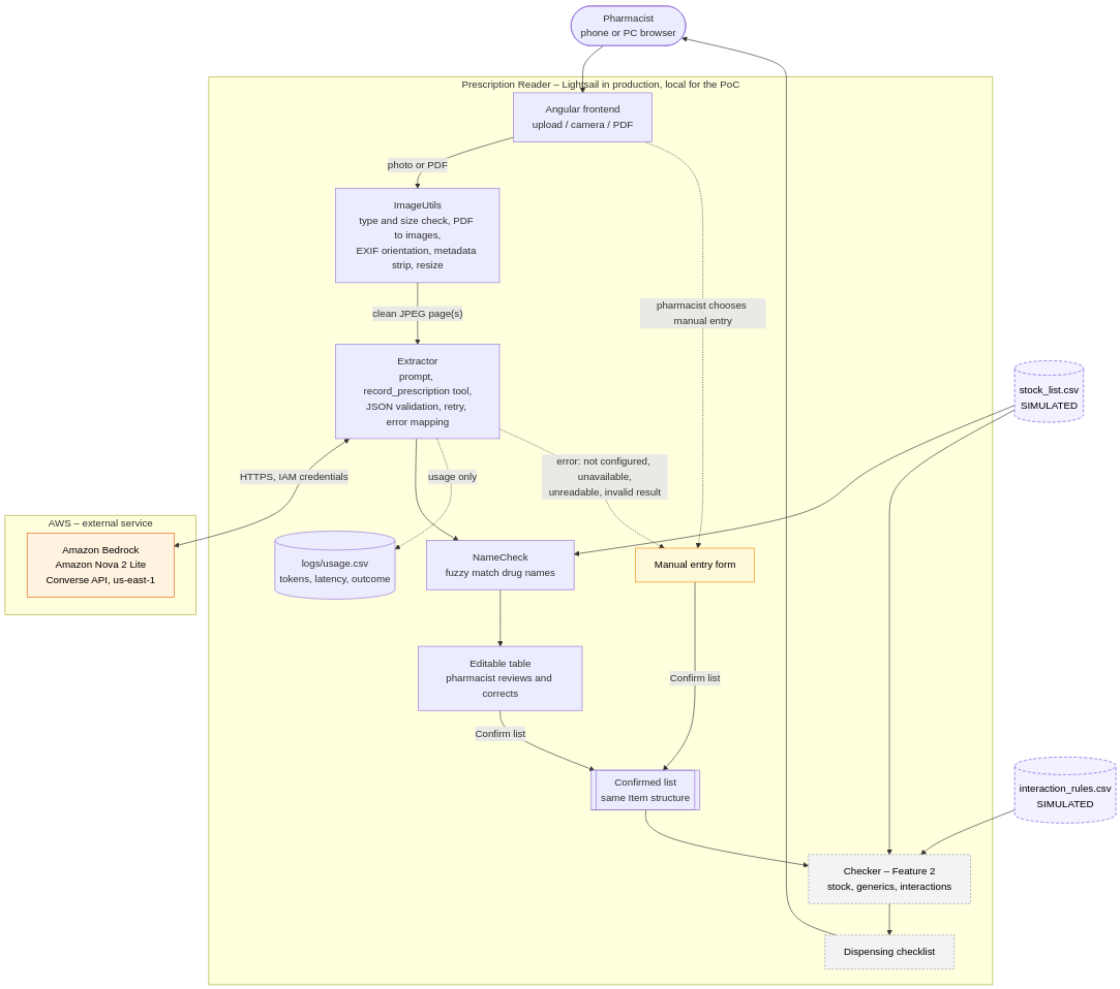
The assignment asks for **two connected features**: Feature 1 (R1, built) produces the confirmed list, and Feature 2 (R2, planned for Week 3) uses that list.

2. The idea and how it works

The "emerging technology" is a **multimodal foundation model** – an AI model that understands both pictures and text. Instead of first turning the picture into raw text (OCR) and then trying to find the drugs in that text, the model looks at the photo and directly returns the items as structured data.

1. The pharmacist takes a photo (or uploads a JPG, PNG, WEBP or PDF) in the web browser.
2. The server checks the file, turns it upright, **removes all hidden information** (such as location data), and makes it smaller.
3. The server sends the clean image to **Amazon Nova 2 Lite** on Amazon Bedrock with instructions (the "prompt").
4. Nova returns each item in a fixed format, with a confidence level: high, medium or low.
5. The server checks the answer and compares each drug name with the pharmacy's stock list.
6. The pharmacist sees the photo and the table side by side, corrects anything wrong, and presses **Confirm list**.

The system in one picture



Architecture (from docs/architecture.md). Orange = external AWS service; purple dashed = simulated data (CSV files); grey dashed = Feature 2, planned; yellow = manual-entry fallback.

How to read it: the pharmacist's browser talks to the app. The app prepares the image and calls Amazon Bedrock. The answer is checked and shown in an editable table. Both the AI path and the manual path end in the same "confirmed list", which Feature 2 will check against the stock list and the interaction rules.

If the AI fails: if anything goes wrong with the AI, the app shows a clear message and opens the **manual entry** form instead, so work never stops.

Alternative that was considered: Amazon Textract (OCR) followed by Amazon Comprehend Medical. It was not chosen because it is two separate steps (reading mistakes flow into the second step) and Comprehend Medical is tuned for American medical text and drug names, so Sri Lankan brands and short forms may be missed.

Where it runs: for this proof of concept everything runs on the student's laptop, and only the Bedrock call leaves the machine. In real use the app would run on one small **AWS Lightsail** server.

3. Technologies used – and why

The AI part (Amazon Web Services)

Technology	What it is, in simple words	Why it was used
Amazon Bedrock	An AWS service that lets an app use AI models over the internet, paying only for what it uses.	The assignment requires AWS services only; no AI server to run.
Amazon Nova 2 Lite	Amazon's AI model that can look at images and read text. Model ID `us.amazon.nova-2-lite-v1:0`.	Reads photos directly into data; cheap; fast. The model can be changed in settings.
Converse API + "tool use"	The way the app talks to Nova. "Tool use" forces Nova to answer in a fixed form (drug name, strength, dose…).	Gives clean, predictable answers instead of free text.
AWS IAM	AWS's system of users and permissions.	A special user that may only call Nova – nothing else (least privilege).
AWS CLI profile	Stores the AWS login keys on the laptop, outside the project.	No password or key is ever in the code or on GitHub.
AWS Budgets	Sends an email if spending passes a limit (USD 5/month).	Protects against a surprise bill.

The app

Technology	What it is, in simple words	Why it was used
Angular 19 (TypeScript)	A framework for building the screens that run in the web browser – the "frontend".	Student's chosen stack; works on PC and phone.
Spring Boot 3.5 (Java 17)	A framework for building the server – the "backend" – that does the work behind the screens.	Student's chosen stack; reliable and well known.
REST API	The set of web addresses (e.g. `/api/scans`) that the screens use to talk to the server.	Keeps the screens and the server separate.
AWS SDK for Java v2	Amazon's official Java library for calling AWS services.	Used to call Bedrock from the server.
Apache PDFBox	A Java library that opens PDF files.	Turns each PDF page into an image for Nova.
TwelveMonkeys, **metadata-extractor**	Image libraries: one opens WEBP images, one reads which way up a phone photo was taken.	Accept more formats; rotate photos correctly.
FuzzyWuzzy (Java)	Compares two words and gives a similarity score (e.g. "Lipitr" vs "Lipitor" = 92%).	Matches drug names to the stock list despite small spelling errors.
Apache Commons CSV	Reads spreadsheet-like CSV files.	Loads the stock list and interaction rules.
Maven, **npm**	Tools that download libraries and build the backend and frontend.	Standard build tools.

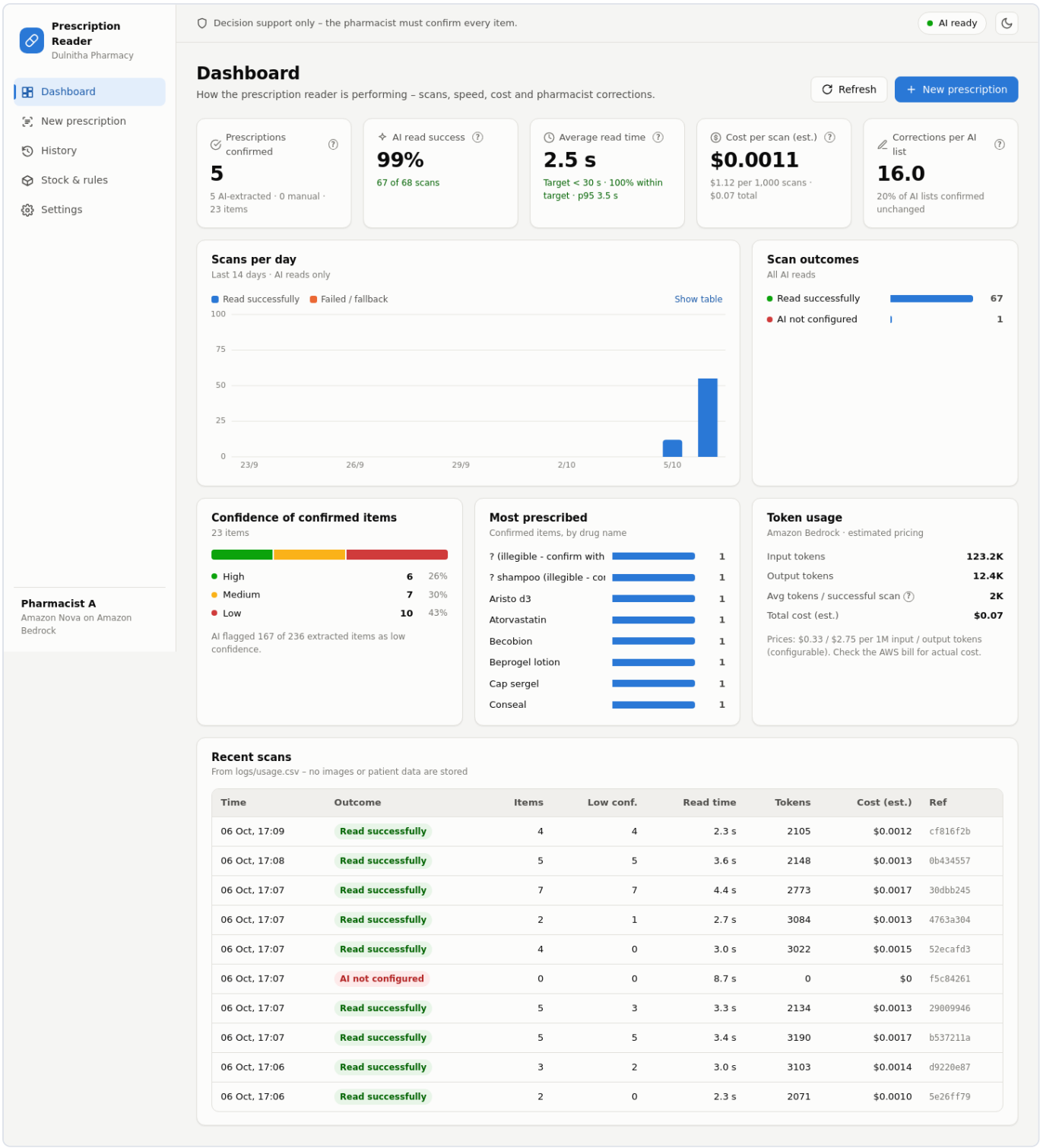
Testing and support

Technology	What it is, in simple words	Why it was used
JUnit 5 + Mockito	Automatic tests for Java. Mockito plays the part of Amazon Bedrock, so tests cost nothing.	62 tests check that every part works.
Python scripts	`cost_estimate.py` works out monthly cost; `evaluate.py` measures reading accuracy.	Evidence for the cost and accuracy sections.
Headless Chrome (Puppeteer)	A web browser controlled by a script, with no window.	Ran the test cases through the real screens and took screenshots.
Mermaid	Draws diagrams from text.	The architecture diagram, which GitHub displays.
Git + GitHub	Keeps the history of every change and stores the code online.	Required: the submission needs a repository link.
Claude Code	An AI coding assistant. It wrote much of the code under the student's direction.	Declared in `docs/ai_use_log.md`. It is **not** part of the running app.

4. The app, page by page

The app has a menu on the left with five pages. At the top of every page is the reminder "*Decision support only – the pharmacist must confirm every item*", a status pill (**AI ready** when AWS is set up) and a button to switch between light and dark mode.

4.1 Dashboard – how the reader is performing

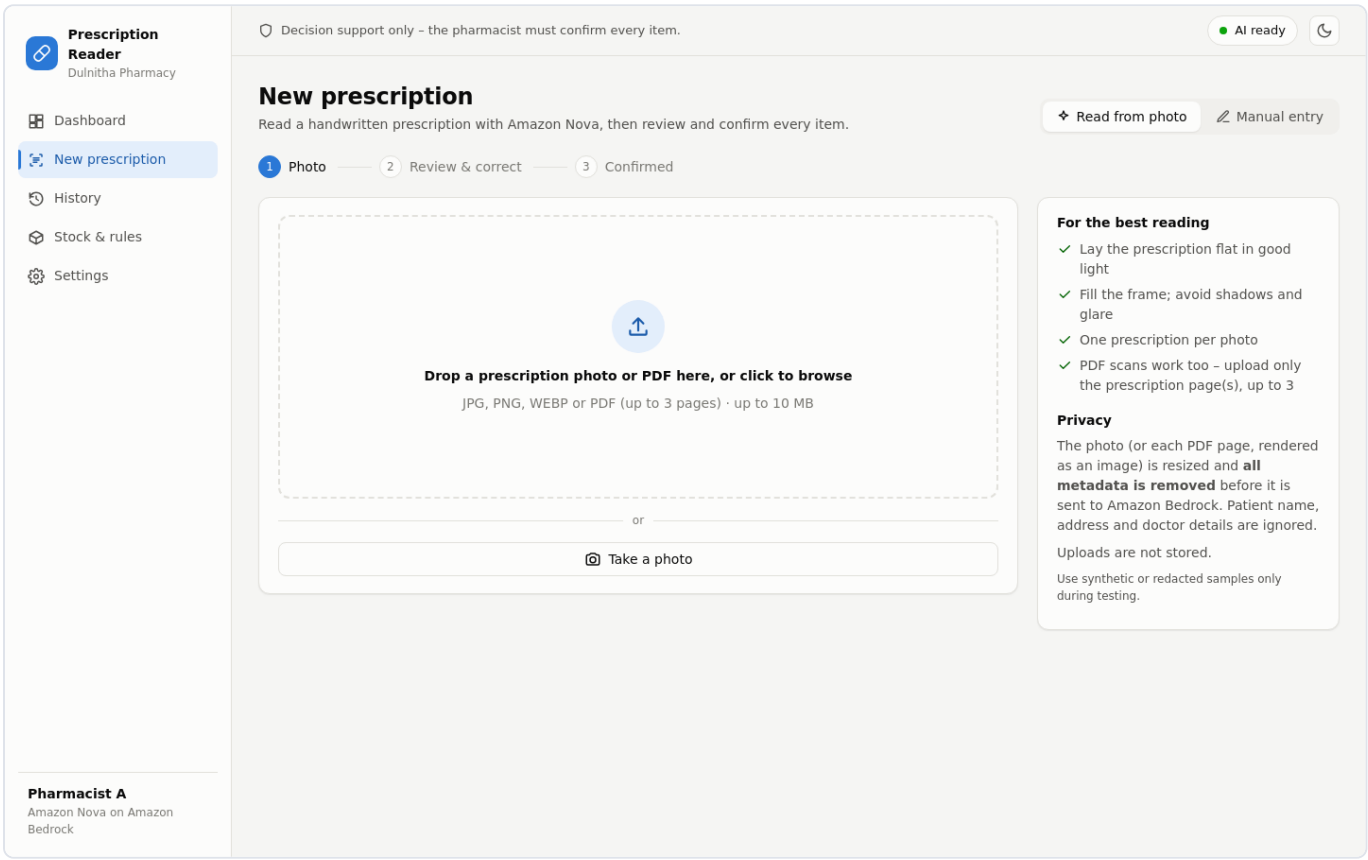


Dashboard with the real figures from testing.

What each part of the dashboard shows

- **Five summary tiles:** lists confirmed; AI read success rate; average read time against the 30-second target; estimated cost per scan (and per 1,000 scans); and how many corrections the pharmacist made per AI list. The **?** icons explain each number when you hover over them.
- **Scans per day:** a bar chart of the last 14 days (blue = read successfully, orange = failed). Hover a bar for exact numbers, or press *Show table*.
- **Scan outcomes:** how many reads succeeded and why the others failed.
- **Confidence of confirmed items:** how many items were high, medium or low confidence.
- **Most prescribed:** the drugs that appear most often.
- **Token usage:** how much of the AI was used and what it cost. ("Tokens" are the units AI services charge by.)
- **Recent scans:** the last 10 reads with time, result, items, read time and cost. No images or patient data are stored.

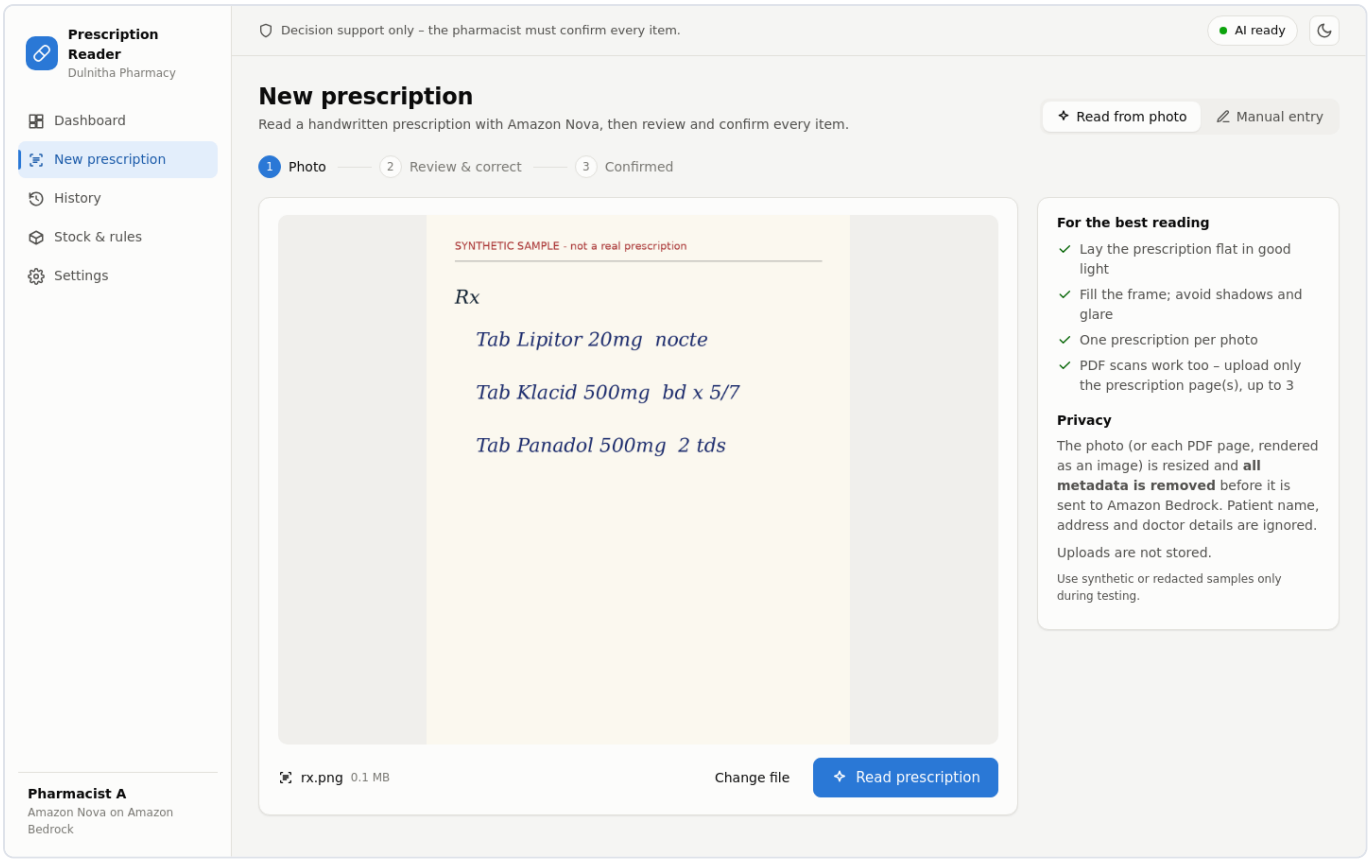
4.2 New prescription – step 1: choose the photo



Drag and drop a photo or PDF, browse for a file, or press "Take a photo" on a phone.

The page shows a three-step progress bar (Photo → Review & correct → Confirmed). At the top right the pharmacist can choose **Read from photo** or **Manual entry**. The right-hand panel gives tips for a good photo and explains the privacy rules.

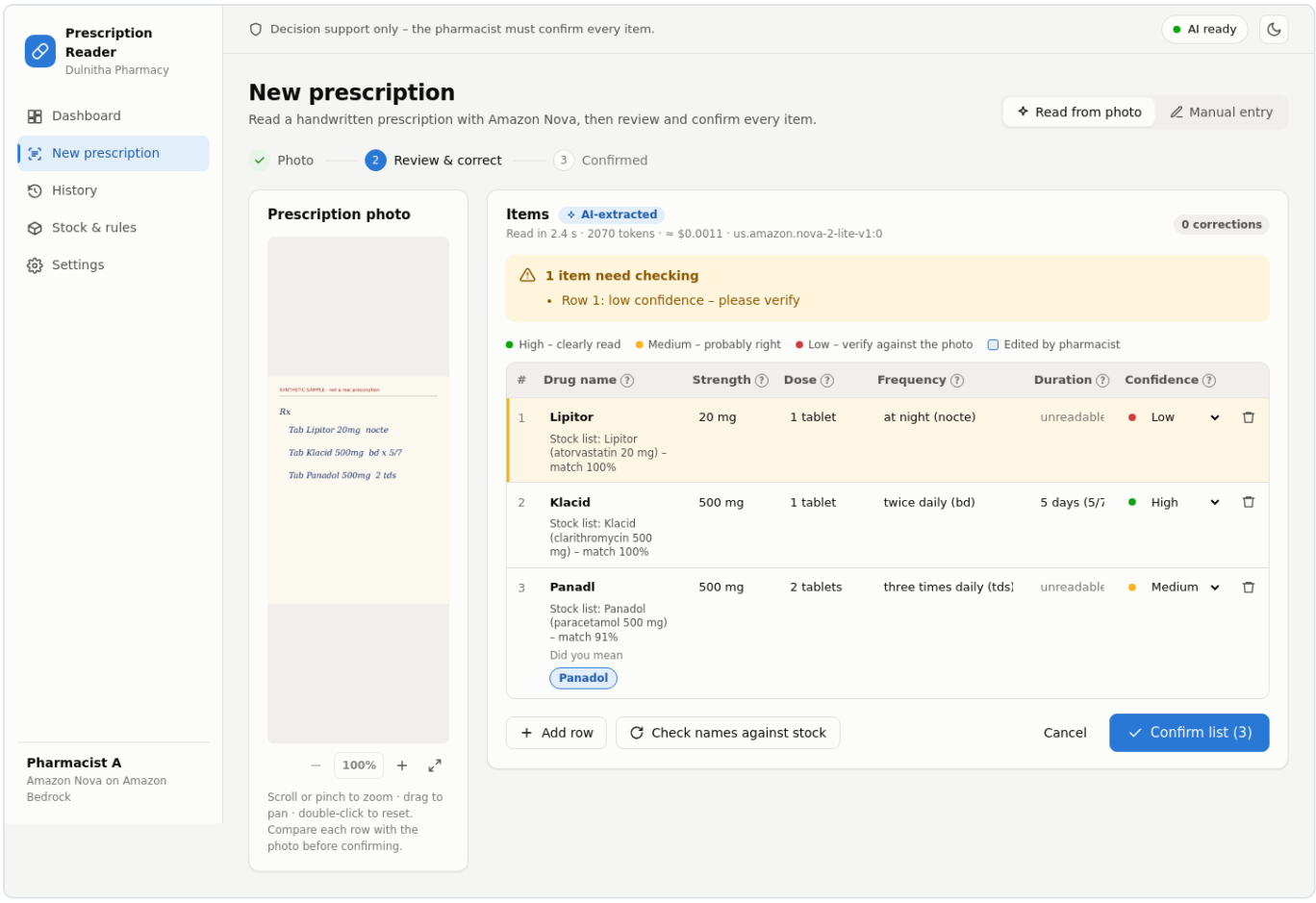
Step 1b: preview and read



After choosing the file, the photo is shown. "Read prescription" sends it to Amazon Nova.

While Nova reads, a scanning line moves over the photo and a timer counts the seconds (target: under 30 s). A PDF (up to 3 pages) is shown page by page. Wrong files are refused straight away with a clear reason – for example "Only JPG, PNG, WEBP or PDF files are accepted" or "The file is larger than 10 MB".

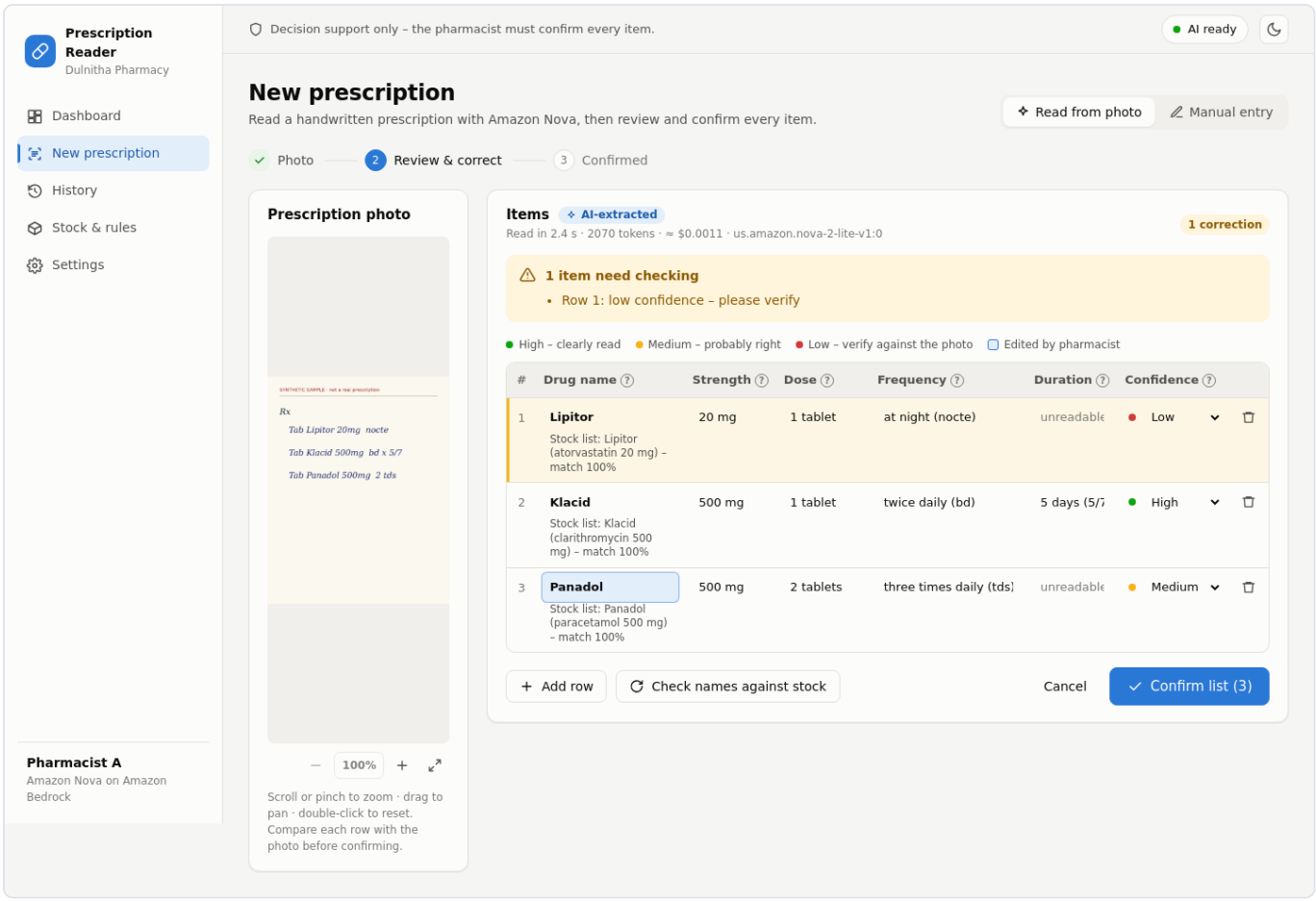
4.3 New prescription – step 2: review and correct



The AI's reading next to the photo. Row 1 is low confidence; row 3 offers a "Did you mean Panadol?" suggestion.

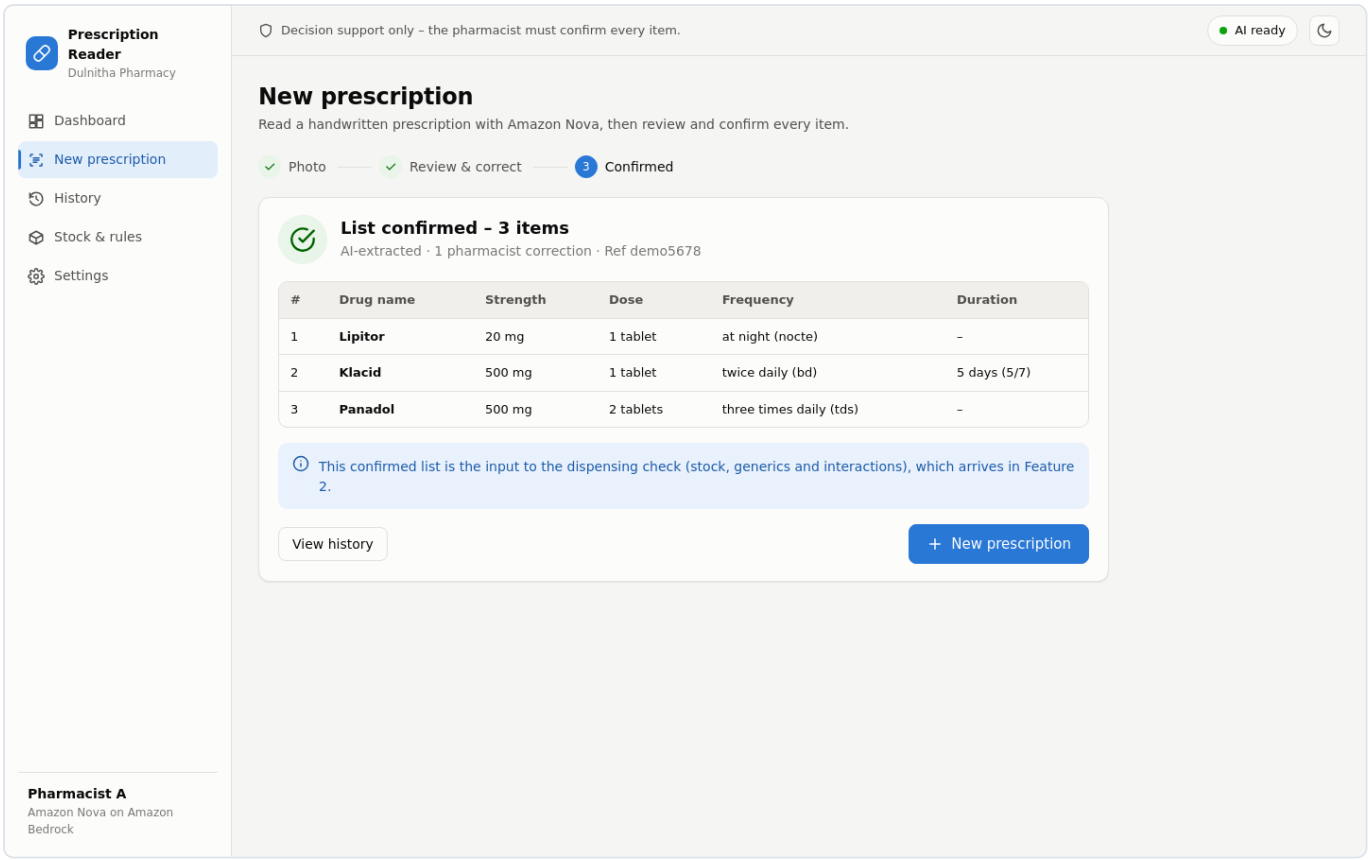
This is the most important screen. Everything is designed so the pharmacist checks the AI rather than trusting it:

- **Photo on the left**, with zoom buttons, scroll-to-zoom and drag-to-move, and a full-screen view – so each row can be compared with the handwriting.
- **Editable table on the right:** drug name, strength, dose, frequency, duration and confidence. Every cell can be changed; rows can be added or deleted.
- **Confidence colours:** High clearly read, Medium probably right, Low verify against the photo. Low rows are highlighted, and a yellow box lists them: "Row 1: low confidence – please verify".
- **Unreadable fields** are left empty and labelled "unreadable" – the AI is told not to guess.
- **Stock-list match** under each drug name, e.g. "Stock list: Klacid (clarithromycin 500 mg) – match 100%". Unknown names say "Not found in stock list – check carefully".
- **"Did you mean…?" chips** suggest close stock-list names; one click applies it (the pharmacist decides).
- **Edited cells turn blue**, and a counter shows how many corrections were made – a measure of how good the AI reading was.
- **Check names against stock** re-runs the name check after edits. **Confirm list** saves the list; it refuses if any row has no drug name.



After the pharmacist corrects "Panadl" to "Panadol": the cell is blue and the counter shows 1 correction.

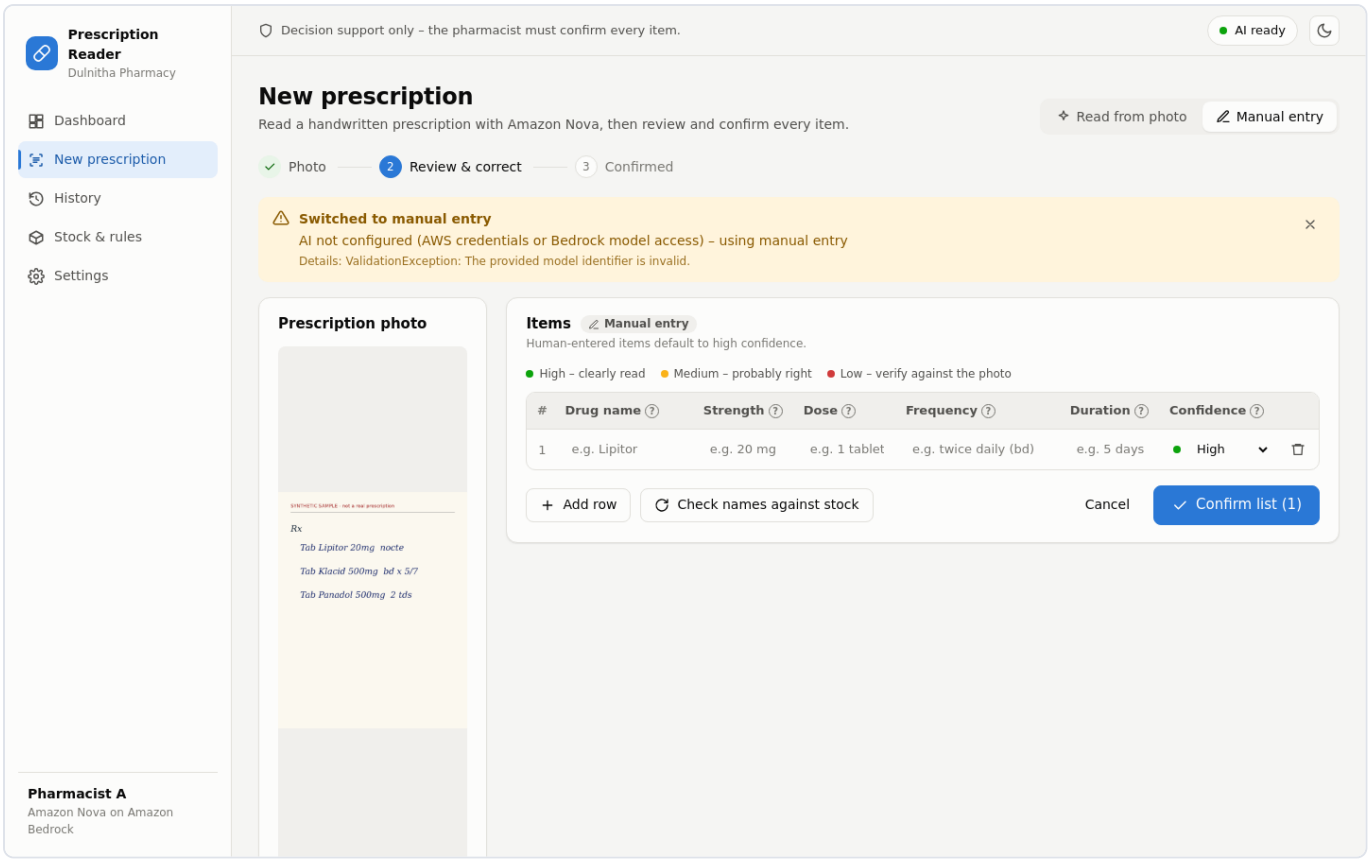
4.4 Step 3: confirmed



The confirmed list, with where it came from (AI-extracted or manual), the number of corrections and a reference number.

This confirmed list is the input for Feature 2 (the dispensing check), which will be built next.

4.5 When the AI fails – automatic manual entry

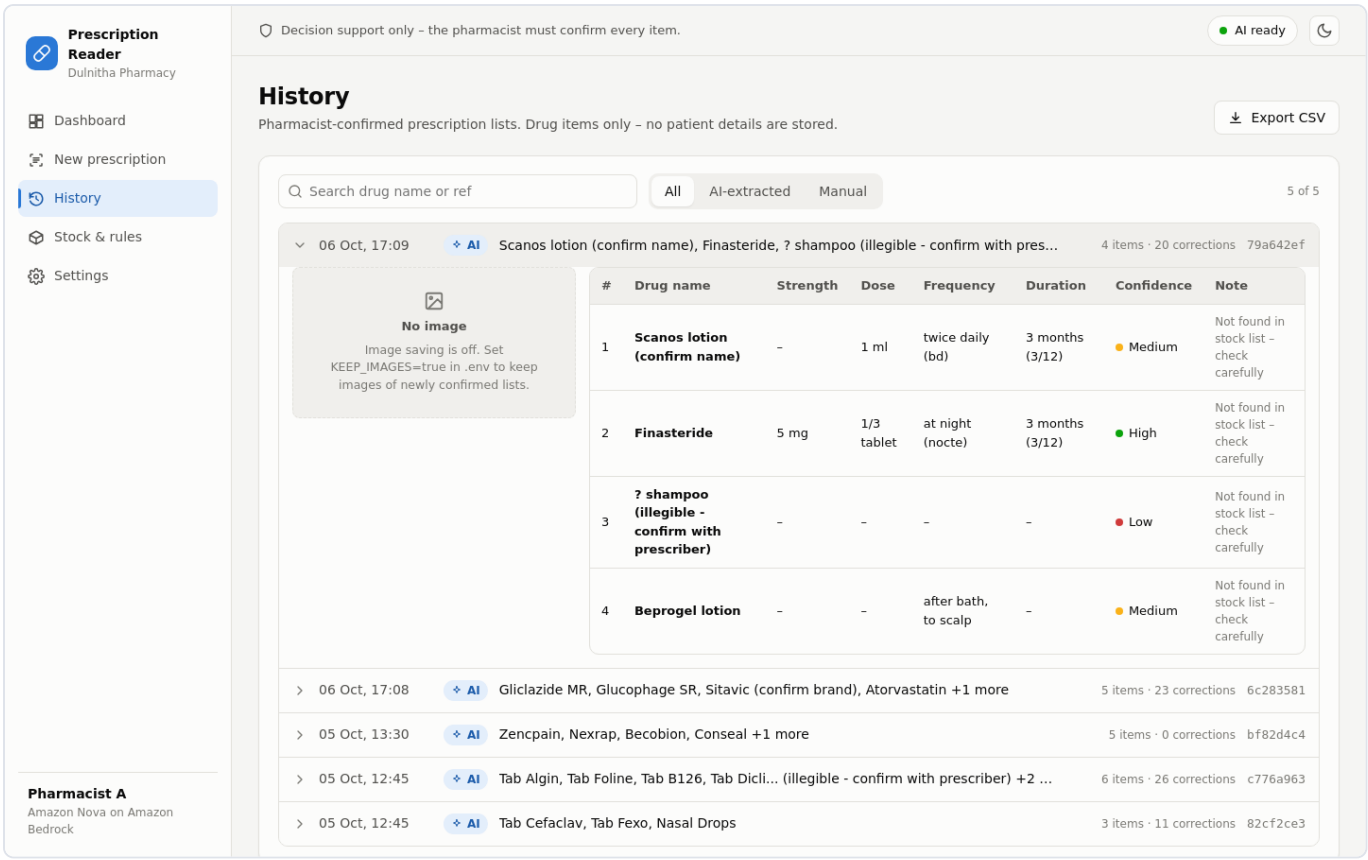


Test case TC3: an invalid model ID. The app explains the problem and opens manual entry, keeping the photo visible.

Every kind of AI problem has its own clear message, and the work can always continue:

Problem	Message the pharmacist sees
AWS not set up, no permission, wrong model	"AI not configured (AWS credentials or Bedrock model access) – using manual entry"
No internet, timeout, AWS busy	"AI service unavailable – using manual entry"
Photo has nothing readable	"Could not read this photo – please retake it, or enter the items manually"
AI answer is broken (after one retry)	"AI returned an unexpected result – using manual entry"

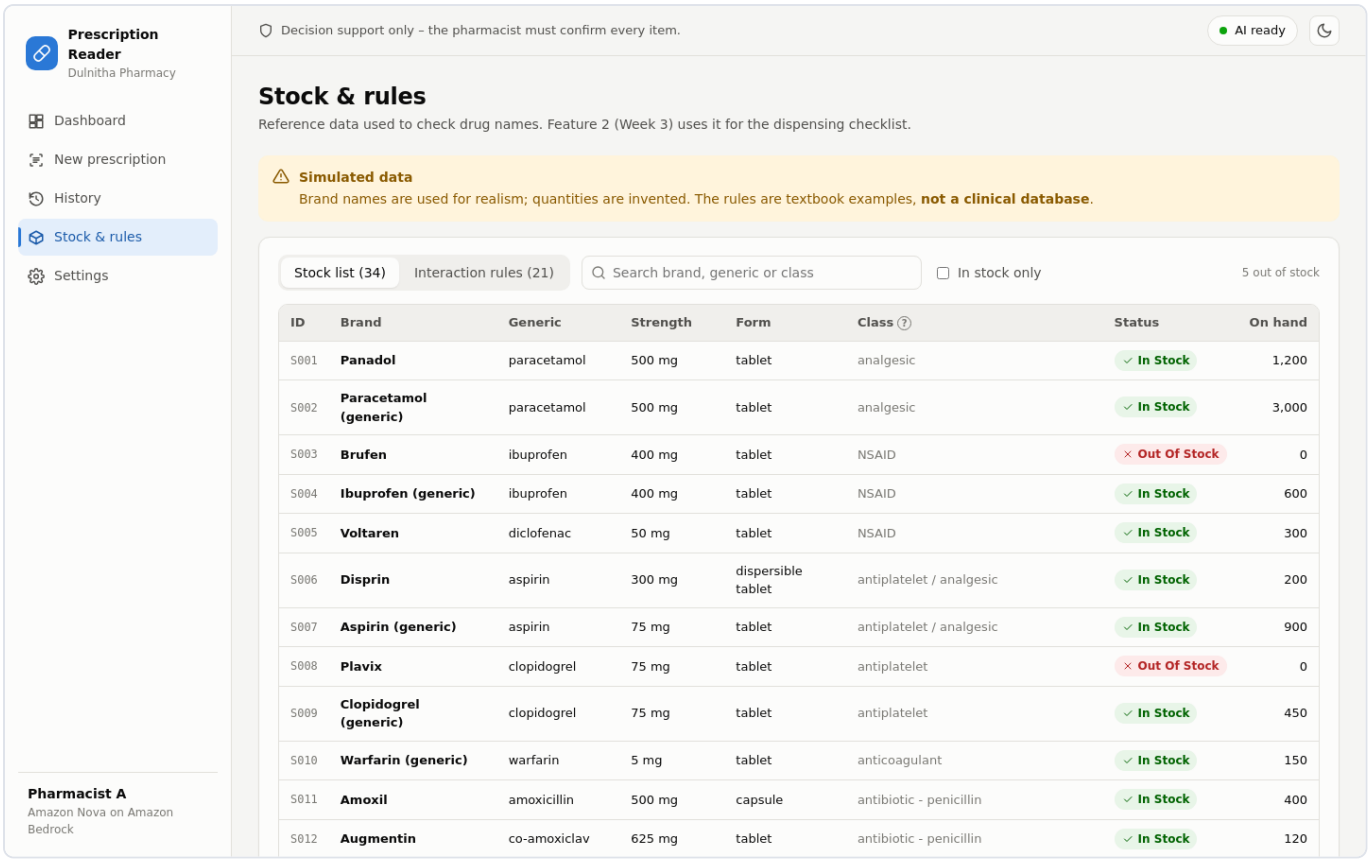
4.6 History – every confirmed list



Confirmed lists from testing. One list is opened to show its items.

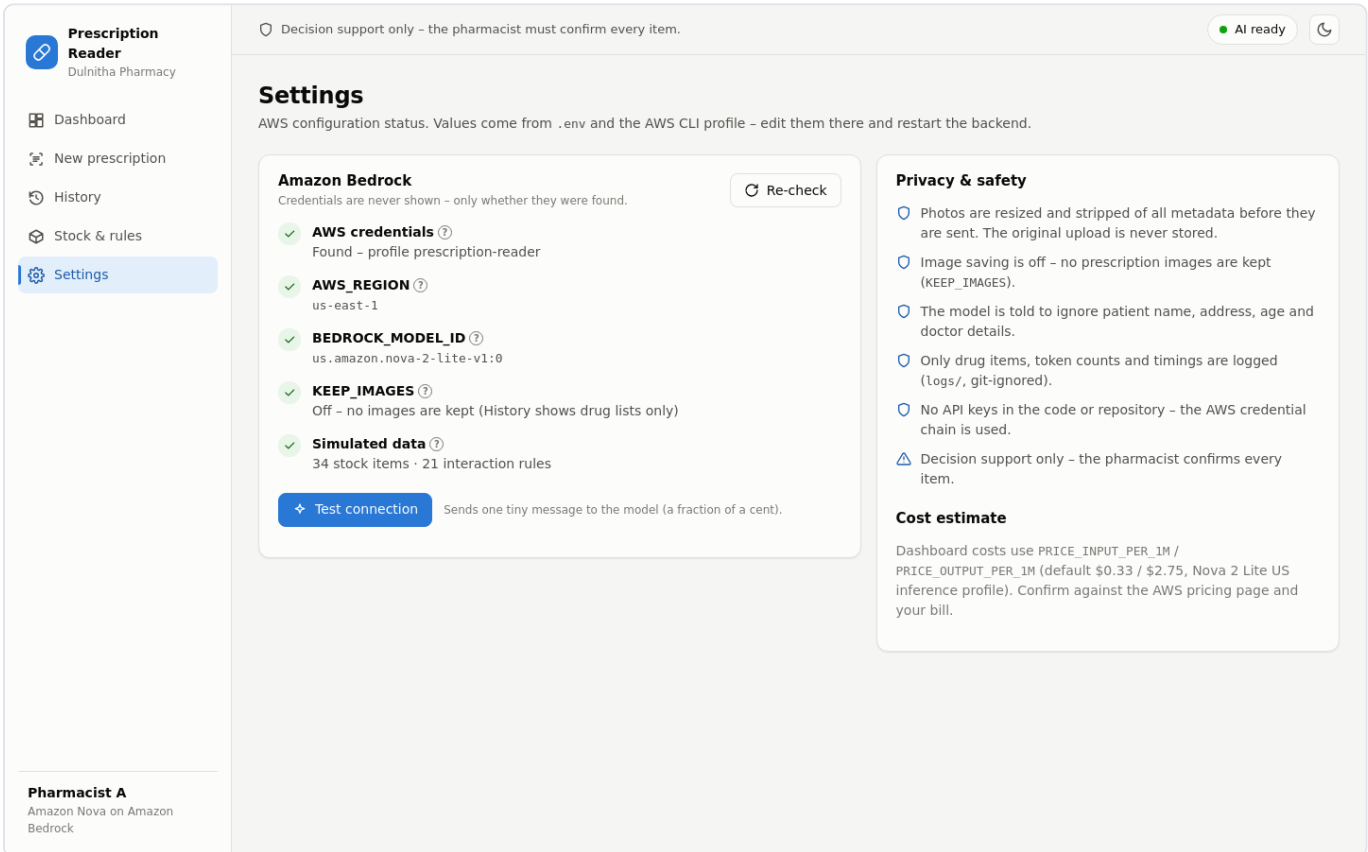
The pharmacist can **search** by drug name or reference, **filter** AI-extracted or manual lists, open a list to see its items, and **export everything to a CSV file** (for example for Excel). Only drug items are stored – never patient details. Keeping the prescription image with a list is **optional and off by default** (`KEEP_IMAGES=true`); when on, images are deleted automatically after 7 days.

4.7 Stock & rules – the reference data



The simulated stock list (34 items) and interaction rules (21 rules), clearly labelled "Simulated data".

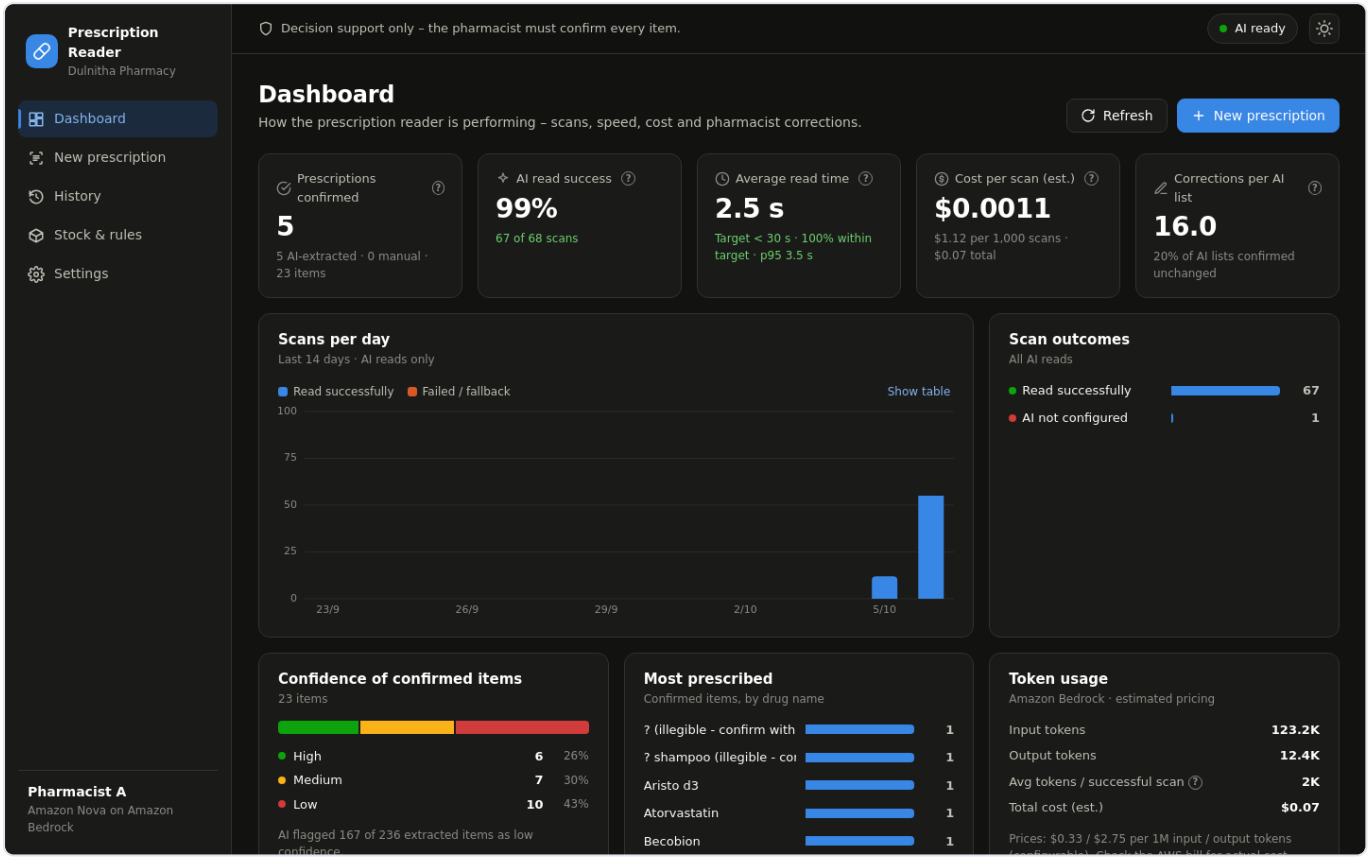
4.8 Settings – is everything connected?



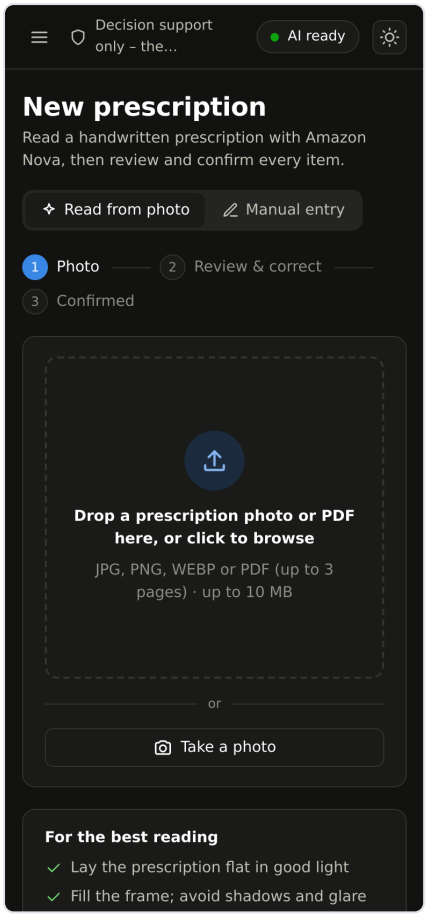
Settings shows whether AWS is set up – never the actual keys – and offers "Test connection".

It shows: whether AWS credentials were found, the region, the AI model, whether images are kept, and the size of the reference data. **Test connection** sends one tiny message to Nova (a fraction of a cent) and reports OK or the exact error with a hint on how to fix it.

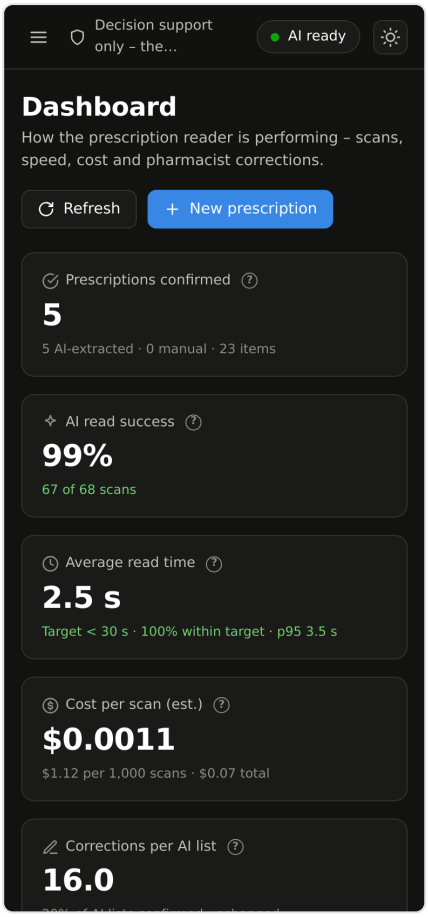
4.9 Dark mode and phones



Dark mode.



Phone: New prescription.



Phone: Dashboard (menu behind the ☰ button).

5. Safety and privacy

Patient safety

- "Decision support only" is shown on every page: the pharmacist confirms every item.
- The AI is told **not to guess** drug names and to leave unreadable fields empty.
- Low-confidence rows and unknown drug names are clearly marked.
- The photo stays next to the table so missed items can be spotted.
- Manual entry is always available.

Privacy (Sri Lanka's PDPA)

- All hidden data in photos and PDFs (location, author…) is removed before sending.
- The AI is told to ignore patient name, address and age and the doctor's details.
- No patient details are stored; images are not kept unless switched on.
- Testing used only redacted prescriptions (names and stamps blacked out), with permission. They are kept off GitHub.
- The AWS user may only call Nova; keys are never in the code.

Still to settle before real use

Images are processed in an AWS region in the USA, because there is no AWS region in Sri Lanka. Sri Lanka's Personal Data Protection Act limits sending health data abroad, so this must be checked, and a login and HTTPS must be added before real patients are involved.

6. Testing and results

6.1 Automatic tests

62 automatic tests, all passing. They check, for example: a good AI answer becomes the right table; a broken answer is retried once; timeouts and permission problems give the right message; wrong or oversized files are refused; PDFs are read; metadata is removed; "Lipitr" matches Lipitor and "Xyzzol" is flagged. Amazon Bedrock is simulated in these tests, so they cost nothing.

6.2 The three required test cases

Test	Input	Result	Verdict
TC1 clear	Redacted Sri Lankan prescription with 5 numbered diabetes and heart medicines	Read in 3.6 s; all 5 items found and all flagged for checking; but names and strengths misread and a duration invented on every item. 23 corrections, then confirmed.	**Partial**
TC2 messy	Redacted Sri Lankan prescription with lotions, a tablet and a shampoo	Read in 2.3 s; all 4 items found and all flagged low; the pharmacist corrected them (20 corrections) and marked an unreadable shampoo "confirm with prescriber".	**Pass**
TC3 error	App started with a wrong AI model name	Clear message, manual entry opened, list typed and confirmed.	**Pass**

6.3 Accuracy test on 9 Sri Lankan prescriptions

The script `evaluate.py` sent 9 redacted Sri Lankan prescriptions (33 items) to the AI and compared the answers with what the prescriptions really say. Three versions of the instructions (prompts v1, v2, v3) and a bigger model (Nova Pro) were compared.

Measure	Prompt v1	Prompt v2 (chosen)	Prompt v3	Nova Pro + v2
Items found	70%	76%	73%	70%
Drug names correct	43%	52%	52%	52%
All fields correct	40%	42%	43%	31%
Wrong items that were flagged	100%	100%	73%	100%
Read time	2.2 s	2.4 s	2.5 s	3.2 s
Cost per scan	USD 0.0010	USD 0.0011	USD 0.0012	USD 0.0026

What this means, in plain words

- **Clear handwriting is read almost perfectly** (for example a list of dressing supplies). **Dense joined-up handwriting is not** – about half the drug names are wrong.
- **The AI is good at knowing when it is unsure.** With prompt v2, every item that had a mistake was marked for checking. This is why it was chosen.
- **The AI sometimes adds things that are not written**, especially a "5 days" duration. Prompt v3 tried to stop this and did not succeed. The pharmacist must check every field.
- **About 1 item in 4 can be missed** without a warning – only comparing with the photo reveals it.
- **A bigger, more expensive model (Nova Pro) did not do better.**
- Changing the instructions made only small differences – the same instructions scored differently from one run to the next. The model's ability to read handwriting is the real limit.

Important limits of this test

Only 9 photos from a small number of patients, so the numbers are indicative, not proof. The "correct answers" were read by the AI assistant and still need to be checked by the student or the pharmacist. The prompts were improved using the same photos they were tested on.

An earlier test on 13 Bangladeshi prescriptions from a public dataset (directions in Bengali) gave similar results: fast, all mistakes flagged, but weak on doses and durations.

7. Cost

From `cost_estimate.py`, using the real token use of 67 scans and assuming 150 handwritten prescriptions a day for 30 days (4,500 scans a month) and USD 1 = LKR 300.

Item	LKR per month	Based on
Amazon Nova 2 Lite (reading 4,500 prescriptions)	1,509	Measured usage × AWS price
AWS Lightsail server	2,100	AWS price (USD 7)
Total running cost	**3,609**	about LKR 0.80 per prescription
One-off setup (installation, training)	25,000 once	Assumption
Value of staff time saved	22,500	Assumption: 1 minute saved per prescription at LKR 300/hour – **not yet measured**

Break-even: the app pays its running cost if it saves only about **10 seconds of staff time per prescription**. Whether it really saves time is not yet known: if the pharmacist has to correct many fields, it may not. This is the main question for a pilot.

8. What is done and what is left

Done

- Problem, requirements, technology choice
- AWS setup (IAM user, model access, budget)
- Feature 1: photo/PDF → AI → editable table → confirm, with manual fallback
- Dashboard, history, stock view, settings, dark mode, phone layout
- 62 automatic tests; TC1–TC3 with screenshots
- Accuracy evaluation on redacted Sri Lankan samples
- Architecture, feasibility draft, cost model, README, AI use log

Still to do

- **Feature 2 – dispensing check** (stock, alternatives, interactions, checklist) – required
- **Timing test**: about 5 prescriptions, typed by hand vs read by the app and corrected
- Pharmacist: written consent, review of the CSVs, check of the "correct answers"
- Commit and push to GitHub
- 4-page report, ZIP file, demo video (≤ 3 minutes)

Overall

The project is on track. The app works and is well tested, and the results are honest: the AI is fast, cheap and safe in the sense that it marks its own mistakes, but it is not accurate enough on dense handwriting to save time without careful checking. Feature 2 is useful whatever the AI's accuracy, because it works on the corrected list.

9. Glossary – simple meanings of technical words

Word	Meaning
AI model / foundation model	A computer program trained on huge amounts of data that can understand and produce text and, for "multimodal" models, images.
Multimodal	Able to handle more than one kind of input – here, pictures and text.
OCR	Optical character recognition: turning a picture of text into plain text, letter by letter, without understanding it.
Prompt	The written instructions sent to the AI together with the image.
Token	A small piece of text or image that AI services count and charge for.
Confidence	How sure the AI says it is about an item: high, medium or low.
Ground truth	What the prescription really says, written by a person, used to mark the AI's answers.
Fallback	The backup path when the main path fails – here, manual entry.
Frontend / backend	The screens in the browser / the server program that does the work.
API	A defined way for two programs to talk to each other, here over web addresses.
Cloud region	The country and data centre where AWS runs the service (here us-east-1, in the USA).
IAM / least privilege	AWS permissions; giving a user only the permissions it really needs.
Metadata	Hidden information inside a file, such as where and when a photo was taken.
Redaction	Blacking out private information, such as names, before a document is used.
Fuzzy matching	Finding words that are similar, not only identical (Lipitr ≈ Lipitor).
Simulated data	Made-up data for testing – here the stock list and interaction rules, which are not a real clinical database.
PDPA	Sri Lanka's Personal Data Protection Act, No. 9 of 2022.
Proof of concept (PoC)	A working first version that shows an idea can work; not yet a finished product.

Prepared with Claude Code (AI coding assistant) from the project's repository, test logs and evaluation on 06/10/2026. Figures marked as assumptions should be confirmed; prices should be checked on the AWS pricing page.